Day – 16 Implementation of Neural Network

Binary Classification of Heart Disease Using Keras Deep Neural Networks

1. Introduction

- Motivation: Heart disease remains one of the leading global causes of mortality. Early, automated detection of cardiovascular risk allows clinicians to intervene early and improve patient outcomes.
- Objective: To design and implement a Multi-Layer Perceptron (MLP) binary classification architecture using TensorFlow/Keras to predict the presence or absence of heart disease based on clinical patient features.

2. Dataset Overview

- Dataset Characteristics: Contains continuous (e.g., age, resting blood pressure, cholesterol levels) and categorical clinical measurements (e.g., chest pain type, fasting blood sugar, slope of peak exercise ST segment).
- Dimensions: Typically structured as $N \times 14$ (or $N \times D$ depending on feature selection), consisting of N patient samples and 13 input features.
- Target Distribution: Binary class target ($y \in \{0,1\}$):
    - Class 0: No heart disease detected.
    - Class 1: Heart disease presence detected.

3. Methodology

- Train-Test Splitting: The dataset is divided using train_test_split with an 80:20 ratio (80% training, 20% testing) to evaluate generalization performance on unseen test data.
- Feature Standardization: Features are rescaled using StandardScaler to have a zero mean ($\mu = 0$) and unit variance ($\sigma = 1$):

$$z = \frac{x - \mu}{\sigma}$$

This prevents high-magnitude features (e.g., serum cholesterol) from dominating gradient updates during training.

4. Model Design

Layer (Type)	Input/Output Shape	Activation Function	Param #	Description
Input	(None, 13)	—	0	Input vector representing 13 scaled patient features
Dense_1	(None, 32)	ReLU	$13 \times 32 + 32 = 448$	First hidden layer extracting feature interactions
Dropout_1	(None, 32)	—	0	Regularization layer (20% dropout rate) to reduce overfitting
Dense_2	(None, 16)	ReLU	$32 \times 16 + 16 = 528$	Second hidden layer refining feature representations
Dense_Output	(None, 1)	Sigmoid	$16 \times 1 + 1 = 17$	Single neuron mapping logits to probability [0,1]

5. Hyperparameters

- Loss Function: Binary Cross-Entropy Loss:

$$\mathcal{L}(y, \hat{y}) = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]$$

- Optimizer: Adam (Adaptive Moment Estimation) with dynamic momentum and learning rate scaling.
- Learning Rate: Default Keras Adam initial learning rate ($\eta = 0.001$).
- Batch Size: 16 samples per mini-batch step.
- Epochs: 50 total training passes over the dataset.

6. Results & Discussion

- Test Accuracy: Evaluates the percentage of total patient records classified correctly across both classes on unseen test data.

- Loss vs. Epoch Analysis: Monitoring validation loss against training loss over the 50 epochs ensures the model converges smoothly without overfitting (mitigated by the Dropout layer).
- Confusion Matrix Evaluation:
    - True Positives (TP): Sick patients correctly diagnosed with heart disease.
    - False Negatives (FN): Sick patients incorrectly classified as healthy.
- Recall Metric:

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

In a clinical context, high recall is prioritized to minimize False Negatives, ensuring potentially high-risk patients are not missed by the model.

```
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
df = pd.read_csv('/content/heart.csv')
X = df.drop(columns=['target'])
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = tf.keras.models.Sequential([
    tf.keras.layers.Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])
model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)
model.fit(X_train, y_train, epochs=50, batch_size=16, validation_data=(X_test, y_test))
```

```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not p
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.7598 - loss: 0.5477 - val_accuracy: 0.7512 -
Epoch 2/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8390 - loss: 0.4161 - val_accuracy: 0.7854 -
Epoch 3/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8561 - loss: 0.3639 - val_accuracy: 0.8146 -
Epoch 4/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8634 - loss: 0.3405 - val_accuracy: 0.8195 -
Epoch 5/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8756 - loss: 0.3132 - val_accuracy: 0.8195 -
Epoch 6/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.8695 - loss: 0.3077 - val_accuracy: 0.8098 -
Epoch 7/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8780 - loss: 0.2958 - val_accuracy: 0.8146 -
Epoch 8/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8854 - loss: 0.2918 - val_accuracy: 0.8146 -
Epoch 9/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8939 - loss: 0.2801 - val_accuracy: 0.8390 -
Epoch 10/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8890 - loss: 0.2907 - val_accuracy: 0.8537 -
Epoch 11/50
```

```python
import numpy as np
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {test_acc * 100:.2f}%\n")
y_pred_probs = model.predict(X_test)
y_pred = (y_pred_probs > 0.5).astype(int)
# 3. Print detailed evaluation metrics
print("--- Confusion Matrix ---")
print(confusion_matrix(y_test, y_pred))
print("\n--- Classification Report ---")
print(classification_report(y_test, y_pred, target_names=['No Disease (0)', 'Disease (1)']))
```

```
Test Accuracy: 95.12%

7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step
--- Confusion Matrix ---
[[ 94   8]
 [  2 101]]
```